

METRONET

Guía de Configuración Inicial

Si estás leyendo este documento es porque vas a comenzar a trabajar en el proyecto METRONET.

Seguí los pasos en orden. No saltees ninguno.

Paso 1 - Crear una cuenta GitHub

1. Ingresar a <https://github.com>
2. Crear una cuenta.
3. Enviar el nombre de usuario GitHub a Logan.
4. Esperar la invitación al repositorio.

Cuando recibas la invitación, aceptarla.

No continúes hasta tener acceso al repositorio.

Paso 2 - Instalar Git

¿Para qué sirve?

Git permite descargar el proyecto y compartir cambios con el resto del equipo.

Descargar desde:

<https://git-scm.com/downloads>

Instalar utilizando las opciones por defecto.

Verificación:

Windows

Abrir PowerShell.

Escribir:

`git --version`

Mac

Abrir Terminal.

Escribir:

```
git --version
```

Resultado esperado:

```
git version x.x.x
```

Si aparece la versión, continuar.

Paso 3 - Instalar Java 17

¿Para qué sirve?

Java es el lenguaje utilizado por el backend.

Descargar desde:

<https://adoptium.net>

Seleccionar:

- Java 17 (LTS)
- Sistema operativo correspondiente

Instalar.

Verificación:

```
java -version
```

Resultado esperado:

```
openjdk version "17"
```

Si aparece una versión 17 o superior, continuar.

Paso 4 - Instalar IntelliJ IDEA Community

¿Para qué sirve?

Es el programa que utilizaremos para trabajar el backend.

Descargar desde:

<https://www.jetbrains.com/idea/download/>

Instalar la versión Community.

Abrir una vez finalizada la instalación para verificar que funciona correctamente.

Paso 5 - Instalar PostgreSQL

¿Para qué sirve?

Es la base de datos del proyecto.

Descargar desde:

<https://www.postgresql.org/download/>

Durante la instalación:

- Mantener puerto 5432.
- Recordar la contraseña configurada.

Verificación:

`psql --version`

Resultado esperado:

psql (PostgreSQL) 16.x

Paso 6 - Instalar DBeaver

¿Para qué sirve?

Permite visualizar y administrar la base de datos mediante una interfaz gráfica.

Descargar desde:

<https://dbeaver.io/download/>

Instalar.

Abrir para verificar funcionamiento.

Paso 7 - Instalar Node.js

¿Para qué sirve?

Permite ejecutar y administrar el frontend.

Descargar desde:

<https://nodejs.org>

Instalar la versión LTS.

Verificación:

node -v

npm -v

Ambos comandos deben devolver un número de versión.

Paso 8 - Instalar Visual Studio Code

¿Para qué sirve?

Será utilizado para trabajar en el frontend y la documentación.

Descargar desde:

<https://code.visualstudio.com>

Instalar.

Abrir para verificar funcionamiento.

Paso 9 - Descargar el Proyecto

Abrir una terminal.

Windows

Abrir PowerShell.

Mac

Abrir Terminal.

Ubicarse en la carpeta donde se desea guardar el proyecto.

Ejecutar:

git clone URL_DEL_REPOSITORIO

Ingresar al proyecto:

cd METRONET-Proyecto

Paso 10 - Abrir el Backend

Abrir IntelliJ IDEA.

Seleccionar:

Open

Elegir:

METRONET-Proyecto/backend

Esperar a que Maven descargue todas las dependencias.

Este proceso puede demorar algunos minutos la primera vez.

Paso 11 - Abrir el Frontend

Abrir Visual Studio Code.

Seleccionar:

Open Folder

Elegir:

METRONET-Proyecto

Paso 12 - Configurar la Base de Datos

Abrir DBeaver.

Crear una conexión PostgreSQL.

Servidor:

localhost

Puerto:

5432

Base de datos:

metronet_proyecto

Usuario:

(consultar configuración del equipo)

Contraseña:

(consultar configuración del equipo)

Verificar que la conexión sea exitosa.

Paso 13 - Verificar Backend

Abrir una terminal.

Ingresar:

cd backend

Ejecutar:

mvn spring-boot:run

Si aparece:

Tomcat started on port 8080

y

Started BackendApplication

la configuración es correcta.

Paso 14 - Verificar Rama de Trabajo

Ejecutar:

git branch

Verificar que se encuentre en su rama correspondiente:

Juan:

desarrollo-juan

Viviana:

desarrollo-viviana

Logan:

desarrollo-logan

No trabajar directamente sobre:

main

o

integracion-equipo

Fin de la Configuración

Si todos los pasos anteriores fueron completados correctamente, el entorno de desarrollo está listo para comenzar a trabajar en el proyecto.

Política de Trabajo con Git

Regla General

Cada integrante del equipo desarrollará sus tareas exclusivamente dentro de su propia rama de trabajo.

Las ramas asignadas son:

- desarrollo-logan
- desarrollo-juan
- desarrollo-viviana

Los desarrolladores no deberán realizar modificaciones directas sobre las ramas:

- main
- integracion-equipo

Integración de Cambios

Una vez finalizada una funcionalidad, cada integrante deberá:

1. Guardar sus cambios mediante commit.
2. Enviar los cambios a GitHub mediante push.
3. Notificar al administrador del proyecto.

El administrador será el único responsable de:

- Revisar los cambios.
- Resolver conflictos de integración.
- Integrar desarrollos en la rama integracion-equipo.
- Realizar pruebas de integración.
- Incorporar cambios a la rama principal main.

Flujo de Trabajo Oficial



Responsabilidades

Desarrolladores

- Trabajar únicamente en su rama personal.
- Realizar commits descriptivos.
- Mantener actualizada su rama.
- No modificar directamente ramas compartidas.

Administrador del Proyecto

- Gestionar el repositorio.
- Aprobar integraciones.
- Mantener la estabilidad de integracion-equipo.
- Mantener la estabilidad de main.
- Coordinar la integración de cambios del equipo.

Objetivo de esta Política

Esta metodología evita:

- Pérdida de código.
- Sobrescritura accidental de cambios.
- Conflictos innecesarios entre desarrolladores.
- Inestabilidad en la versión principal del proyecto.

Además permite que todos los integrantes trabajen simultáneamente sin afectar el trabajo de los demás.